

Software Requirements Specification

for Medifly

Version 1.0

Prepared by Medifly Team:

Muhammad Zayan (2023554)	Product Owner / System Architect
Muhammad Ahmad Amjad (2023361)	Scrum Master / Drone Expert
Abdullah Ejaz Janjua (2023038)	AI Lead / Developer

December 14, 2025

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Document Conventions	3
1.3	Intended Audience and Reading Suggestions	3
1.4	Product Scope	3
1.5	References	4
2	Overall Description	4
2.1	Product Perspective	4
2.2	Product Functions	4
2.3	User Classes and Characteristics	4
2.4	Operating Environment	5
2.5	Design and Implementation Constraints	5
2.6	Assumptions and Dependencies	5
3	External Interface Requirements	5
3.1	User Interfaces	5
3.2	Hardware Interfaces	6
3.3	Software Interfaces	6
3.4	Communications Interfaces	6
4	System Features	6
4.1	Mission Creation and Configuration	6
4.1.1	Description and Priority	6
4.1.2	Functional Requirements	6
4.2	Autonomous Flight Navigation (A*)	7
4.2.1	Description and Priority	7
4.2.2	Functional Requirements	7
4.3	Real-Time Monitoring	7
4.3.1	Description and Priority	7
4.3.2	Functional Requirements	7
5	Other Nonfunctional Requirements	7
5.1	Performance Requirements	7
5.2	Safety Requirements	7
5.3	Security Requirements	8
5.4	Software Quality Attributes	8
6	Appendices	8
6.1	Appendix A: Team Responsibilities	8
6.2	Appendix B: Project Feasibility Notes	8
6.3	Appendix C: Risk Analysis	8

1 Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for **Medifly**, an autonomous drone-based medicine supply delivery solution. This SRS covers the requirements for the software components including flight control, computer vision-based obstacle avoidance, **A* (A-Star)** pathfinding, real-time GPS tracking, and the mobile application interface.

1.2 Document Conventions

- **REQ-n**: Requirement Identifier.
- ~~SRS~~: ~~Software Requirements Specification~~.
- ~~SR~~: ~~System Requirement (from backlog)~~.
- ~~UR~~: ~~User Requirement (from user stories)~~.
- Priorities are defined as High, Medium, or Low based on the Fibonacci series (13, 8, 5, etc.) used in the team's backlog estimation.

1.3 Intended Audience and Reading Suggestions

- **Developers (Internal)**: For implementing the Python scripts, AI models, and Flutter app.
- **Stakeholders (Government/NGOs)**: To understand the product scope and limitations.
- **Testers**: To validate requirements against the "Definition of Done".

1.4 Product Scope

Medifly is an autonomous drone solution designed to deliver medical aid to remote and inaccessible areas, specifically targeting the Northern Areas of Pakistan (e.g., Gilgit).

- **Primary Goal**: Timely delivery of medical supplies to calamity-hit or remote regions.
- **Key Capabilities**: Live route planning using A*, obstacle avoidance via computer vision, and solar-assisted battery recharging.
- **Limitations**: The system is not designed for the generic public; it is strictly for Government collaboration in Phase 1.

This project is meant to serve as a capstone project for software engineering course at UI&I

1.5 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- Medifly Feasibility Analysis (Economic, Technical, Legal).
- Project Backlog (Sprint 1-3) Scrum Notes.

2 Overall Description

2.1 Product Perspective

Medifly is a self-contained product that integrates proprietary software with off the shelf hardware. It acts as a bridge between high-level mission planning (Mobile App) and low-level flight execution (Drone Hardware).

- **Hardware Stack:** Pixhawk Flight Controller, Jetson Orin Nano, Karrot Camera, GPS, Telemetry.
- **Software Stack:** Ardupilot Firmware, Python Action Scripts, Flutter Mobile App, Firebase Database.

↳ move to 2.4

2.2 Product Functions

The major functions of the system include:

- **Automated Flight:** Autonomous takeoff, navigation, and payload drop.
- **Smart Navigation:** Utilizing the **A* Algorithm** to chart the fastest and safest route.
- **Obstacle Avoidance:** Real-time detection of obstacles using a custom trained Object Detection Model.
- **Energy Management:** Monitoring the 20,000 mAh Li-Ion battery and utilizing solar panels for auto-recharging.
- **Mission Management:** User authentication and mission configuration via the mobile app.
- **Live Tracking:** Real-time telemetry display (Location, Battery, Status) on the ground station.

2.3 User Classes and Characteristics

- **Government/NGO Relief Employees (End Users):**
 - **Characteristics:** Non-technical users requiring a simple interface.
 - **Training:** Will require a "crash course" on drone safety and system usage.
- **Internal Developers/Admins:**
 - **Characteristics:** Technical experts responsible for PID tuning, model training, and maintenance.

2.4 Operating Environment

- **Geography:** Remote, mountainous terrain (Northern Areas).
- **Weather:** Operable in conditions that are not "harsh" (moderate wind/rain allowed; extreme storms restricted).
- **Connectivity:** Requires Telemetry/Radio link for drone-to-ground communication.

→ Add hardware & tech stack along with version number here.

2.5 Design and Implementation Constraints

- **Hardware Constraints:**
 - Must run on Jetson Orin Nano (Onboard) and Pixhawk.
 - Drone Frame: Carbon Fibre (Non-VTOL structure).
 - Development Machine: Intel HX CPU, Nvidia GPU (12GB VRAM).
- **Budget Constraint:** Total hardware acquisition budget is capped at approx. 4 Lakh PKR.
- **Time Constraint:** Project completion within 10-12 months.
- **Legal:** Design is Intellectual Property (IP); requires drone operation license.

2.6 User Documentations: we will deliver comprehensive documentation of how to use this along with drone + app

2.6 Assumptions and Dependencies

- **Assumption:** Weather conditions will not be severe enough to ground the drone permanently.
- **Assumption:** The 20,000 mAh battery + solar panel will provide sufficient endurance for the target range.
- **Dependency:** Availability of GPS satellites for navigation.

3 External Interface Requirements

3.1 User Interfaces

The user interface is a mobile application developed in Dart and Flutter.

- **Dashboard:** Displays drone health, battery status, and connectivity.
- **Map View:** Shows live drone icon on a map with the A* generated path overlay.
- **Mission Planner:** Form inputs for destination coordinates and medicine payload details.

Where is error message standard?

Standard buttons, function that will appear on screen.

3.2 Hardware Interfaces

- **Camera:** Interface with Karrot Camera for visual input to the Jetson Nano.
- **Flight Controller:** MAVLink communication with Pixhawk/Ardupilot.
- **Power:** Interface with Battery Management System (BMS) for the 20k mAh Li-Ion battery and solar input.
- **Sensors:** GPS, Telemetry Radio, Video Transmitter/Receiver.

3.3 Software Interfaces

- **Database:** Firebase for storing user profiles, flight logs, and CAD data.
- **AI Engine:** Python scripts running Object Detection Models on the Jetson Nano.
- **Communication:** Interface between the Detection Model and Action Scripts to trigger flight maneuvers.

3.4 Communications Interfaces

- **Telemetry:** Radio frequency communication for real-time status updates.
- **Video Link:** Dedicated video transmitter/receiver for visual feed.
- **Data Security:** Encrypted communication to ensure data privacy across different NGO customers.

4 System Features

4.1 Mission Creation and Configuration

4.1.1 Description and Priority

Allows authorized users (Government/NGO) to define flight parameters. **Priority: High (13 Story Points)**

4.1.2 Functional Requirements

- **REQ-1:** The system shall allow users to log in securely using Firebase authentication.
- **REQ-2:** The system shall allow users to set a destination point on the map.
- **REQ-3:** The system shall allow users to input medicine payload details.
- **REQ-4:** The system shall allow configuration of PID values (Admin only).

4.2 Autonomous Flight Navigation (A*)

4.2.1 Description and Priority

Core flight logic using A* algorithm and computer vision. **Priority: High (13 Story Points)**

4.2.2 Functional Requirements

- **REQ-5:** The system shall calculate the fastest route using the **A* (A-Star)** pathfinding algorithm.
- **REQ-6:** The system shall detect obstacles using the onboard Object Detection Model.
- **REQ-7:** The system shall automatically trigger evasion scripts upon obstacle detection.
- **REQ-8:** The drone shall autonomously drop the payload at the destination without manual intervention.

4.3 Real-Time Monitoring

4.3.1 Description and Priority

Live tracking for the user. **Priority: Medium (8 Story Points)**

4.3.2 Functional Requirements

- **REQ-9:** The app shall display the drone's live GPS location.
- **REQ-10:** The app shall show Estimated Time of Arrival (ETA).
- **REQ-11:** The app shall display a summary of locations visited and current drone condition.

5 Other Nonfunctional Requirements

5.1 Performance Requirements

- **Latency:** Real-time tracking updates must have a latency of less than 2 seconds.
- **Accuracy:** GPS positioning must be accurate within a 2-meter radius for safe delivery.

5.2 Safety Requirements

- **Failsafe (RTL):** The drone must Return to Launch (RTL) if the connection is lost or battery is critical.
- **Structural Integrity:** The Carbon Fibre frame must withstand standard operating loads and minor impacts.

- **Weather Safety:** The system must warn against flying in weather conditions that exceed the drone's physical capabilities.

5.3 Security Requirements

- **Data Privacy:** Customer data must be isolated; one agency cannot view another's mission logs.
- **Access Control:** Strict authentication to prevent unauthorized mission launches.

5.4 Software Quality Attributes

- **Usability:** The app must be intuitive for non-technical relief workers.
- **Maintainability:** Modular code structure (Python scripts separate from App logic) to allow independent updates.

6 Appendices

6.1 Appendix A: Team Responsibilities

- **Muhammad Zayan:** Product Owner, Full Stack App Dev (UI/UX, DBMS).
- **Muhammad Ahmad Amjad:** Scrum Master, Drone Manufacturing, Hardware Integration.
- **Abdullah Ejaz Janjua:** AI Expert, Python Scripting, Detection Model Development.

6.2 Appendix B: Project Feasibility Notes

- **Budget:** 4 Lakh PKR (Hardware).
- **Operational Cost:** 10-20k PKR/month.
- **Database Cost:** 20-30k PKR.
- **Timeline:** 10-12 Months (1 Month Requirements, 2 Months Design, 4 Months Dev, 2 Months Testing, 1.5 Months Deployment/Maintenance).

6.3 Appendix C: Risk Analysis

- **GPS Failure:** Risk of mid-flight navigation loss.
- **Motor Failure:** Risk of crash due to hardware malfunction.
- **Debris Damage:** Structural damage from weather or foreign objects.

Appendix C: Analysis Models

[Placeholder for class diagram]